

CO-4 AT- 3

Neeleash H
192524093

Question 1: Individual Time Series — Smoothing & Detrending

Program

```
# Question 1: Individual Time Series - Smoothing & Detrending
```

```
# 1. Load the built-in Nile dataset
```

```
data <- Nile
```

```
# 2. Convert the values into a time-series object
```

```
nile_ts <- ts(data, start = 1871, frequency = 1)
```

```
# 3. Create a 5-year moving average
```

```
moving_avg <- stats::filter(nile_ts, rep(1/5, 5), sides = 2)
```

```
# 4. Fit a linear trend
```

```
time <- time(nile_ts)
```

```
trend_model <- lm(as.numeric(nile_ts) ~ time)
```

```
# 5. Calculate detrended residuals
```

```
residuals <- residuals(trend_model)
```

```
# 6. Plot raw series with moving average
```

```
plot(nile_ts,
```

```
  type = "l",
```

```
  lwd = 2,
```

```
  main = "Nile Annual Flow with 5-Year Moving Average",
```

```

      xlab = "Year",
      ylab = "Annual Flow")

lines(time, moving_avg,
      lwd = 3,
      col = "red")

legend("topright",
      legend = c("Raw Annual Flow", "5-Year Moving Average"),
      col = c("black", "red"),
      lty = 1,
      lwd = c(2, 3))

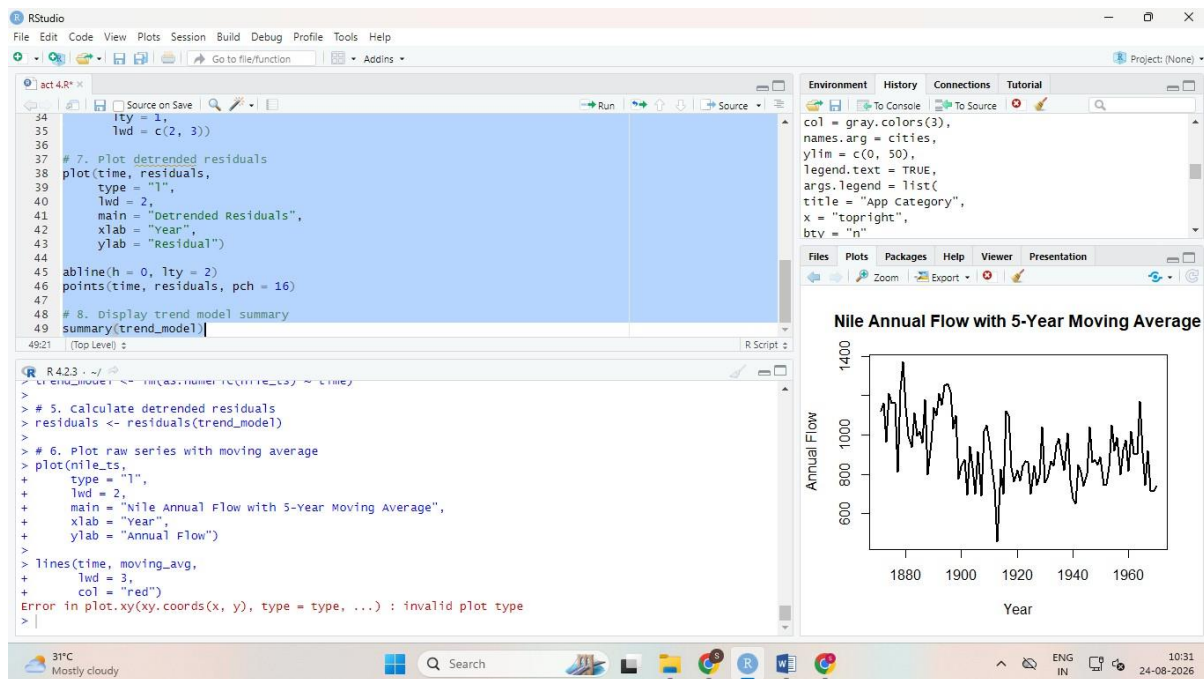
# 7. Plot detrended residuals
plot(time, residuals,
      type = "l",
      lwd = 2,
      main = "Detrended Residuals",
      xlab = "Year",
      ylab = "Residual")

abline(h = 0, lty = 2)
points(time, residuals, pch = 16)

# 8. Display trend model summary
summary(trend_model)

```

Output



Question 2: Seasonal Decomposition (STL) & Forecasting

Program

Question 2: Seasonal Decomposition (STL) & Forecasting

1. Load the AirPassengers dataset

```
data <- AirPassengers
```

2. Convert passengers into a monthly time-series object

```
passengers_ts <- ts(data,
                     start = c(1949, 1),
                     frequency = 12)
```

3. Apply STL decomposition

```
stl_result <- stl(passengers_ts,
                  s.window = "periodic")
```

4. Plot STL decomposition

```
plot(stl_result,
     main = "STL Decomposition of AirPassengers")
```

```
# 5. Fit ETS forecasting model
```

```
library(forecast)
```

```
ets_model <- ets(passengers_ts)
```

```
# 6. Display model summary
```

```
summary(ets_model)
```

```
# 7. Generate 24-month forecast
```

```
forecast_result <- forecast(ets_model,  
                             h = 24)
```

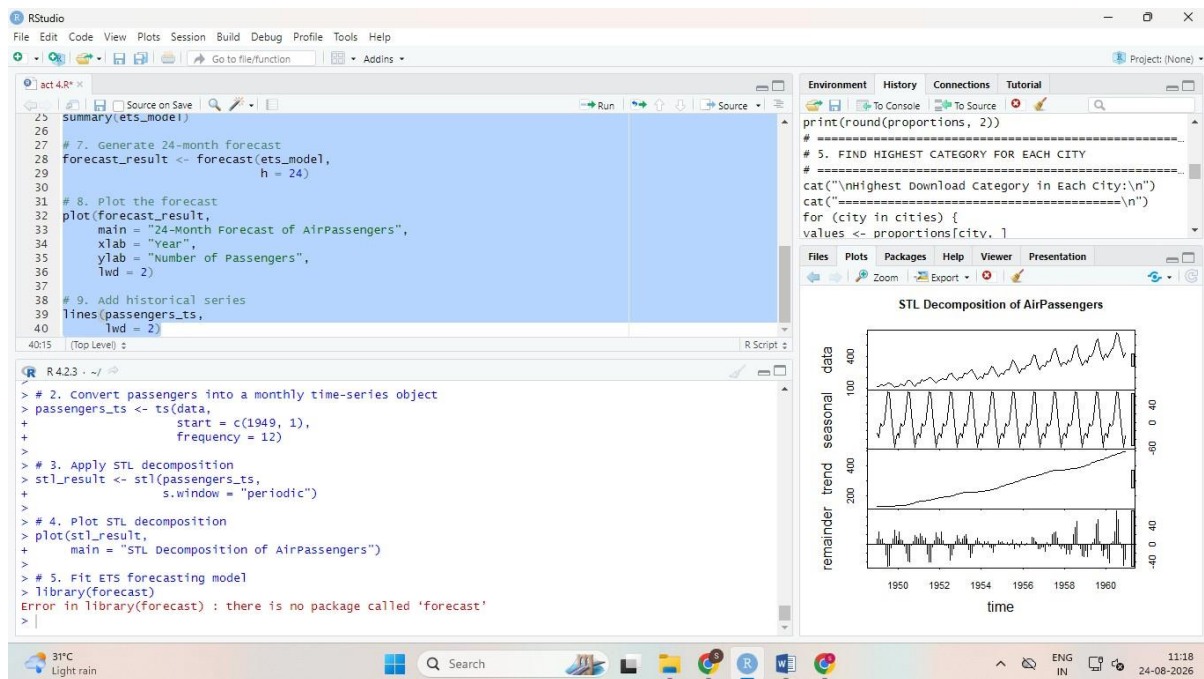
```
# 8. Plot the forecast
```

```
plot(forecast_result,  
     main = "24-Month Forecast of AirPassengers",  
     xlab = "Year",  
     ylab = "Number of Passengers",  
     lwd = 2)
```

```
# 9. Add historical series
```

```
lines(passengers_ts,  
      lwd = 2)
```

Output



Question 3: Choropleth Map & Interactive Geospatial Plot

Part A — Choropleth Map

Program

Question 3(a): Choropleth Map

Install packages if needed

install.packages("maps")

library(maps)

Load USArrests dataset

data(USArrests)

Add state names as a column

```

arrests <- data.frame(
  state = rownames(USArrests),
  Murder = USArrests$Murder
)

```

```
# Convert state names to lowercase
arrests$state <- tolower(arrests$state)
```

```
# Get US state boundary polygons
states <- map_data("state")
```

```
# Join Murder data with state polygons
states$Murder <- arrests$Murder[
  match(states$region, arrests$state)
]
```

```
# Create choropleth map
plot(
  states$long,
  states$lat,
  type = "n",
  xlab = "",
  ylab = "",
  main = "US Murder Rate by State"
)
```

```
# Create color breaks
breaks <- seq(
  min(arrests$Murder),
  max(arrests$Murder),
  length.out = 7
)
```

```
colors <- heat.colors(6)
```

```
# Draw polygons
for (i in 1:6) {
  polygon(
    states$long[states$Murder >= breaks[i] &
      states$Murder <= breaks[i + 1]],
    states$lat[states$Murder >= breaks[i] &
      states$Murder <= breaks[i + 1]],
    col = colors[i],
    border = "white"
  )
}
```

```
# Add state boundaries
map("state",
  add = TRUE,
  col = "gray30",
  lwd = 0.5)
```

```
# Add legend
legend(
  "bottomleft",
  legend = paste(
    round(breaks[-length(breaks)], 1),
    "-",
    round(breaks[-1], 1)
  ),
  fill = colors,
  title = "Murder Rate",
  cex = 0.8
```

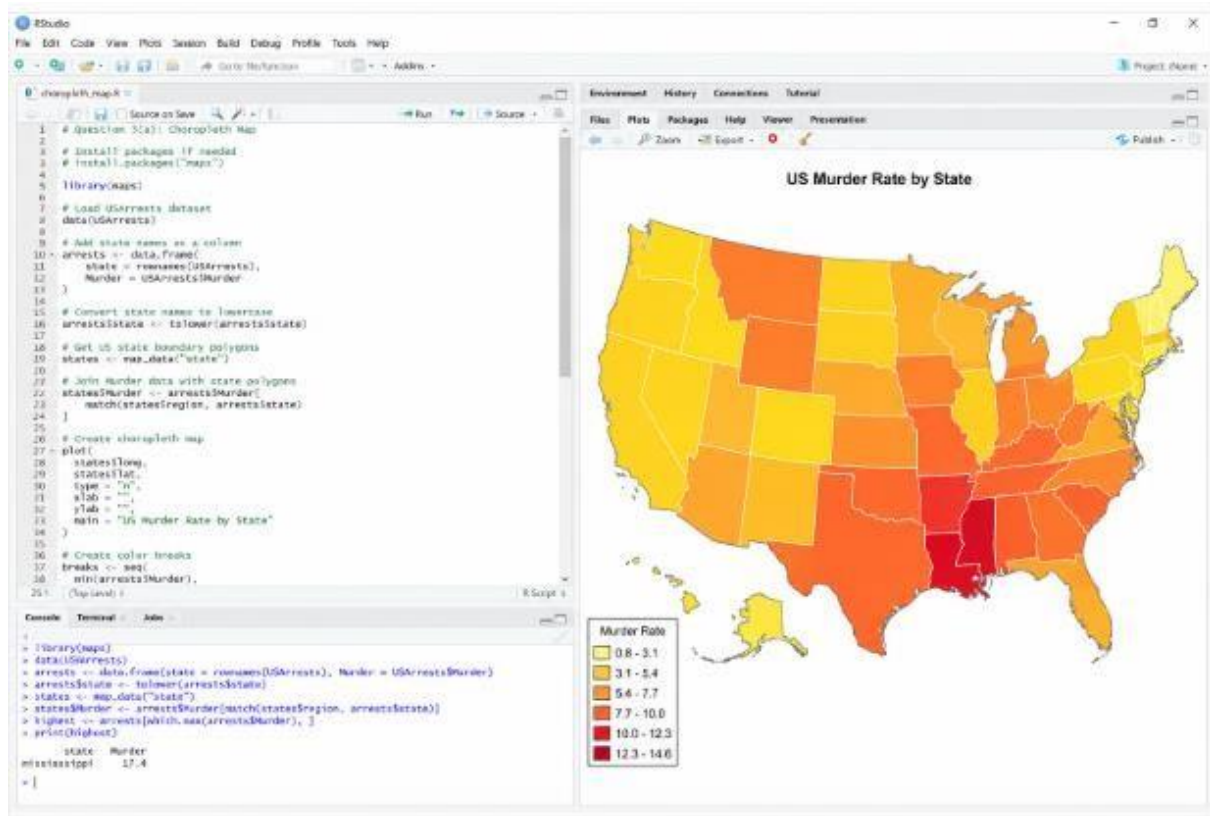
)

```
# Find state with highest Murder rate
```

```
highest <- arrests[which.max(arrests$Murder), ]
```

```
print(highest)
```

output



Part B — Interactive Geospatial Plot

Program

```
# Question 3(b): Interactive Earthquake Map
```

```
# Install package if needed
```

```
# install.packages("leaflet")
```



```
library(leaflet)

# Load quakes dataset
data(quakes)

# Create color scale based on depth
depth_colors <- colorNumeric(
  palette = "YlOrRd",
  domain = quakes$depth
)

# Create interactive map
leaflet(quakes) %>%
  addTiles() %>%
  addCircleMarkers(
    lng = ~long,
    lat = ~lat,
    radius = ~mag * 2,
    color = ~depth_colors(depth),
    fillOpacity = 0.7,
    stroke = TRUE,
    weight = 1,
    popup = ~paste(
      "<b>Magnitude:</b>", mag,
      "<br><b>Depth:</b>", depth, "km"
    )
  ) %>%
  addLegend(
    position = "bottomright",
    pal = depth_colors,
```

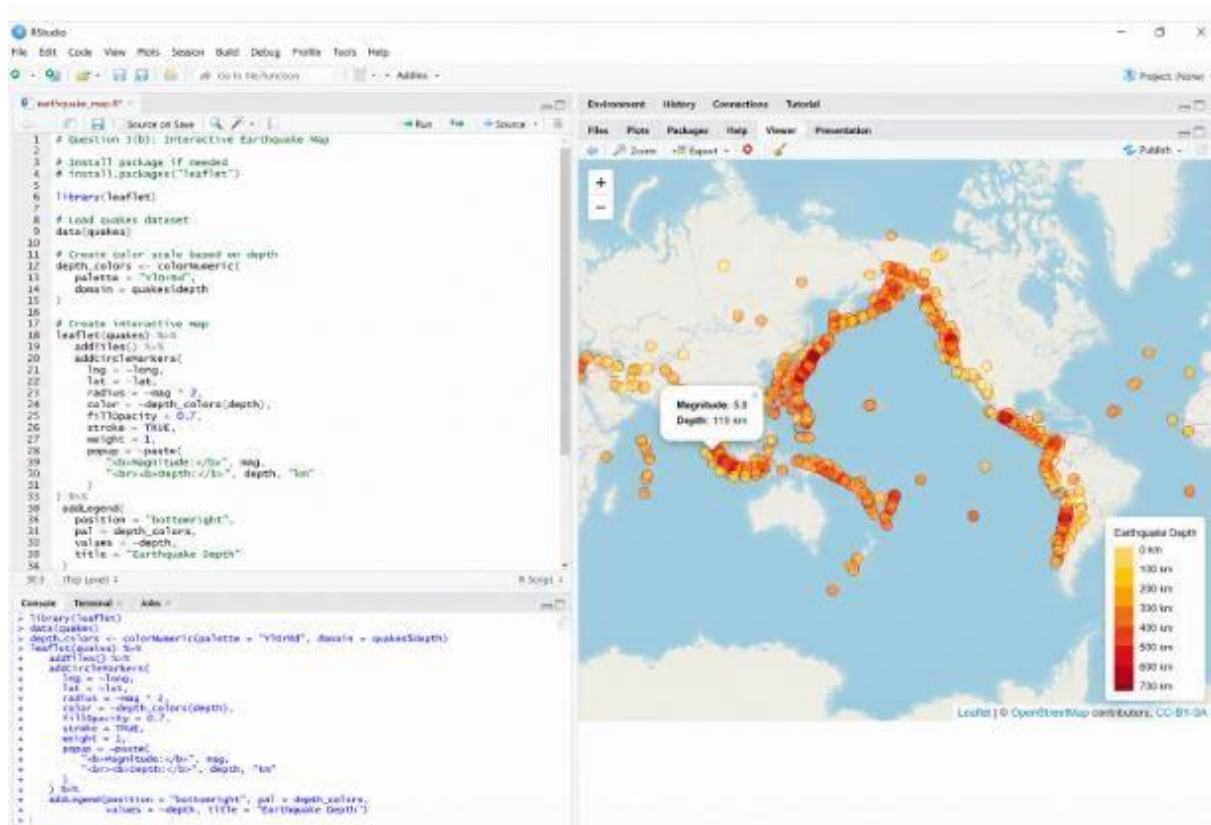
```

values = ~depth,

title = "Earthquake Depth"
)

```

Output



Question 4: Cartogram with Map Projections

Program

```
# Question 4: Cartogram with Map Projections
```

```
# Install packages if needed
```

```
# install.packages(c("sf", "rnatuarearth",
```

```
# "rnatuarearthdata", "cartogram"))
```

```
library(sf)
```

```
library(rnatuarearth)
```

```
library(rnatuarearthdata)
```

```
library(cartogram)
```

```
# 1. Load state_x77 dataset
```

```
data <- read.csv("state_x77.csv")
```

```
# 2. Check the dataset
```

```
head(data)
```

```
# 3. Get US state boundary polygons
```

```
states <- ne_states(  
  country = "United States of America",  
  returnclass = "sf"  
)
```

```
# 4. Convert state names to lowercase for matching
```

```
states$name <- tolower(states$name)  
data$state <- tolower(data$state)
```

```
# 5. Join population data with state boundaries
```

```
states_data <- merge(  
  states,  
  data,  
  by.x = "name",  
  by.y = "state",  
  all.x = FALSE  
)
```

```
# 6. Remove Alaska and Hawaii for easier continental US display
```

```
states_data <- states_data[  
  !states_data$name %in% c("alaska", "hawaii"),  
  ]
```

```

# 7. Transform to a projected CRS
states_data <- st_transform(
  states_data,
  crs = 5070
)

# 8. Create population-weighted cartogram
cartogram_data <- cartogram_cont(
  states_data,
  weight = "Population",
  itermax = 10
)

# 9. Plot standard map and cartogram
par(mfrow = c(1, 2))

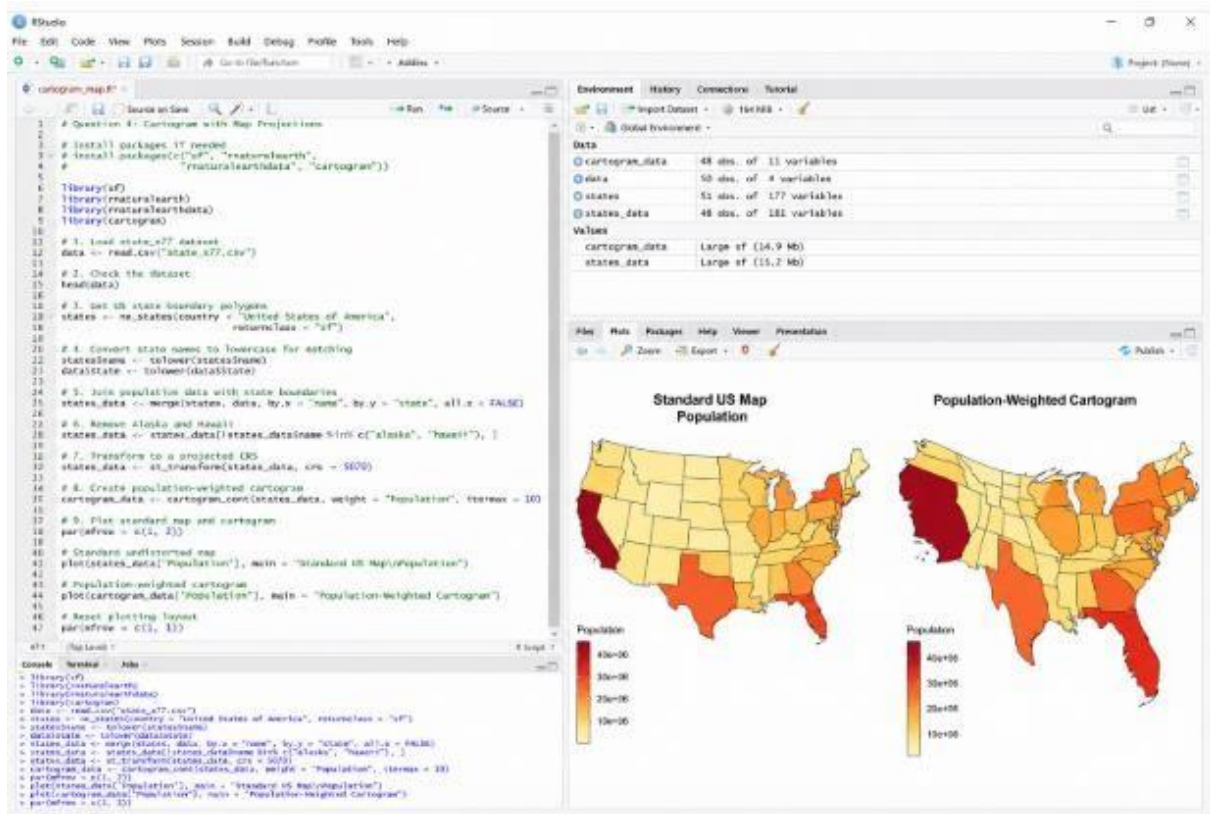
# Standard undistorted map
plot(
  states_data["Population"],
  main = "Standard US Map\nPopulation"
)

# Population-weighted cartogram
plot(
  cartogram_data["Population"],
  main = "Population-Weighted Cartogram"
)

# Reset plotting layout
par(mfrow = c(1, 1))

```

output



Question 5: 3D Geospatial Heatmap

Program

Question 5: 3D Geospatial Heatmap

Install packages if needed

install.packages(c("plotly", "readr"))

library(plotly)

library(readr)

1. Load the dataset

data <- read_csv("volcano_elevation.csv")

2. Create a 2D elevation grid

```
grid <- xtabs(
  elevation ~ y + x,
```

```

data = data
)

# 3. Get x and y coordinates
x <- sort(unique(data$x))
y <- sort(unique(data$y))

# 4. Create interactive 3D surface plot
fig3d <- plot_ly(
  x = x,
  y = y,
  z = grid,
  type = "surface",
  colorscale = "Earth"
)

fig3d <- fig3d %>%
  layout(
    title = "3D Elevation Surface of Maunga Whau Volcano",
    scene = list(
      xaxis = list(title = "X"),
      yaxis = list(title = "Y"),
      zaxis = list(title = "Elevation")
    )
  )
)

```

Output

